



CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

DISCIPLINA: ENGENHARIA DE SOFTWARE

RELATÓRIO DO PROJETO: CORTEJÁ - SISTEMA DE AGENDAMENTO PARA BARBEARIAS

Integrantes:

- GABRIEL HENRIQUE MENDONÇA BATISTA
- JOÃO DANIEL MEYER PITTA MACEDO
- JULIO RODRIGO DOS SANTOS
- LEANDRO JESUS DOS SANTOS
- LUAN CARLOS COSTA FERREIRA
- LUIZ ANSELMO COSTA FERREIRA

2. INTRODUÇÃO

2.1 O que é o sistema desenvolvido

O **Cortejá** é uma plataforma digital (web/mobile) desenvolvida para otimizar o fluxo de atendimento em barbearias e salões de beleza. O sistema permite que clientes agendem serviços de forma autônoma, escolhendo o profissional, o horário e o serviço desejado, enquanto fornece aos administradores ferramentas de gestão de agenda e controle de serviços.

2.2 Qual problema ele resolve

Atualmente, muitas barbearias ainda dependem de agendamentos manuais via telefone ou mensagens de texto, o que resulta em:

- Conflitos de horários (double booking);
- Esquecimento de compromissos por parte dos clientes;
- Interrupção constante do trabalho do barbeiro para atender chamadas;
- Dificuldade na gestão financeira e de produtividade da equipe.

O Cortejá elimina esses gargalos através de um fluxo automatizado e centralizado.

2.3 Objetivo do projeto

O objetivo principal é desenvolver um sistema robusto e intuitivo que facilite a conexão entre barbeiros e clientes, garantindo uma gestão eficiente do tempo e uma experiência de usuário moderna e ágil.

3. DESCRIÇÃO DOS USUÁRIOS (ATORES)

No sistema Cortejá, identificamos dois atores principais:

Ator	Papel no Sistema
Cliente	Realiza o cadastro no sistema, visualiza a disponibilidade dos profissionais, seleciona serviços (corte de cabelo, barba, etc.) e confirma o agendamento. Pode também cancelar ou reagendar horários.

Ator	Papel no Sistema
Barbeiro / Administrador	Gerencia sua própria agenda, define horários de trabalho, cadastra novos serviços e preços, e visualiza o histórico de atendimentos realizados. O administrador também tem acesso a relatórios de faturamento.

4. CASOS DE USO

Abaixo, listamos os principais casos de uso que compõem a lógica de negócio do sistema:

1. **UC01 - Realizar Cadastro:** O cliente ou barbeiro cria uma conta fornecendo dados básicos.
2. **UC02 - Agendar Serviço:** O cliente seleciona o serviço, o profissional e o horário disponível.
3. **UC03 - Gerenciar Agenda:** O barbeiro visualiza, confirma ou altera os agendamentos do dia.
4. **UC04 - Cancelar Agendamento:** Permite que ambas as partes cancelem o horário com antecedência mínima definida.

Diagrama de Casos de Uso (Descrição): O diagrama apresenta o ator **Cliente** conectado aos casos de uso de Cadastro, Agendamento e Cancelamento. O ator **Barbeiro** está conectado ao Gerenciamento de Agenda e Cadastro de Serviços.

5. DIAGRAMA DE CLASSES

O modelo de dados segue a lógica de Orientação a Objetos para garantir escalabilidade.

5.1 Principais Classes

- **Classe Usuario:** Classe base contendo `id`, `nome`, `email` e `telefone`.
- **Classe Agendamento:** Relaciona um `Cliente`, um `Barbeiro` e um `Servico`, contendo atributos como `data`, `hora` e `status`.
- **Classe Servico:** Define o tipo de atendimento, contendo `nomeServico`, `preco` e `duracaoEstimada`.

5.2 Atributos e Métodos

- **Agendamento:**
 - *Atributos:* `private LocalDateTime dataHora, private String status.`
 - *Métodos:* `confirmarAgendamento(), cancelar().`

6. ENCAPSULAMENTO

6.1 Como foi aplicado

No projeto Cortejá, o encapsulamento foi aplicado para proteger a integridade dos dados sensíveis. Todos os atributos das classes de modelo foram definidos como `private`. O acesso e a modificação desses dados ocorrem exclusivamente através de métodos públicos `getters` e `setters`.

6.2 Exemplos Práticos

Na classe `Agendamento`, o atributo `status` não pode ser alterado diretamente para evitar que um agendamento seja marcado como "concluído" sem passar pela validação de pagamento.

```
private String status;

public String getStatus() {

    return this.status;

}

public void setStatus(String novoStatus) {

    if (validarTransicao(novoStatus)) {

        this.status = novoStatus;

    }

}
```

6.3 Importância no sistema

Isso garante que regras de negócio sejam respeitadas (por exemplo, um preço não pode ser negativo) e facilita a manutenção do código, pois mudanças internas na classe não afetam quem a utiliza de fora.

7. USO DO SCRUM

7.1 Organização da Equipe

A equipe foi dividida seguindo os papéis do Scrum: um integrante atuou como **Scrum Master**, outro como **Product Owner**, e os demais como **Development Team**.

7.2 Sprints

O projeto foi executado em 4 Sprints de uma semana cada:

- **Sprint 1:** Levantamento de requisitos e criação dos diagramas de Casos de Uso.
- **Sprint 2:** Modelagem do Banco de Dados e Diagrama de Classes.
- **Sprint 3:** Desenvolvimento do Backend e implementação do encapsulamento.
- **Sprint 4:** Finalização da interface e testes de integração.

7.3 Dificuldades Encontradas

A maior dificuldade foi a conciliação da lógica de horários (evitar sobreposição de agendamentos no mesmo profissional) e a gestão de tempo da equipe durante as Sprints curtas.

8. DESAFIOS E SOLUÇÕES

- **Desafio:** Sincronização de horários em tempo real.
- **Solução:** Implementação de uma trava de banco de dados (locking) que impede que dois clientes selecionem o mesmo horário simultaneamente durante o checkout.

9. CONCLUSÃO

9.1 Aprendizados

O projeto permitiu consolidar conhecimentos de UML, Padrões de Projeto e Metodologias Ágeis. A transição da teoria para a prática mostrou como a documentação prévia economiza tempo de codificação.

9.2 Pontos Positivos e Melhorias

- **Positivo:** A interface ficou intuitiva e o fluxo de agendamento funcional.
- **Melhoria:** No futuro, pretendemos implementar um sistema de pagamentos online e notificações via WhatsApp.

9.3 Pergunta Reflexiva

Como o uso de diagramas e do encapsulamento ajudou no desenvolvimento do projeto?

O uso de diagramas (Casos de Uso e Classes) serviu como um mapa para a equipe, evitando ambiguidades sobre o que deveria ser construído. Já o encapsulamento permitiu que o desenvolvimento fosse modular; pudemos alterar a lógica interna de uma classe sem "quebrar" outras partes do sistema, garantindo segurança e organização ao código.

Referências

1. Documento de Diretrizes do Projeto de Engenharia de Software.
2. <https://youtu.be/JQSsqMCVi1k?si=EAaAujAB98GSsDUb>
3. <https://youtu.be/OTO1MBMmH9g?si=CqQoNPZ0SsJHDSZK>
4. Normas ABNT para Trabalhos Acadêmicos.